

ft_paragraph

Hot off the press

Summary:
In this exercise, you will lay out a text into lines and paragraphs of a given size, using a structure to store your layout settings.

Version: 1.0

Contents

I Introduction **2**

II Common Instructions **3**

III AI Instructions **4**

IV Mandatory part **5**

IV.1 Technical considerations 5

IV.2 The structure. 6

IV.3 Expected output. 7

IV.4 Too much, or not enough. 7

IV.5 Examples. 8

V Submission and peer-evaluation **9**

Chapter I

Introduction

Mainz, 1450. In the workshop of Johannes Gutenberg, the brand new printing press works day and night.

Every page is composed by hand: an apprentice picks the lead letters one by one and lines them up in a *composing stick*, a small tool that can only hold a fixed number of characters. Full lines are then grouped into blocks, and blocks are laid out on the page.

You have just been hired as an apprentice typesetter, and your master is extremely strict about layout:

- every line must contain exactly the same number of characters;
- every paragraph must contain exactly the same number of lines;
- every page must contain exactly the same number of paragraphs.

When there is not enough text to fill a page, he wants to know precisely what is missing. When there is too much, he wants to see what is left.

Sorting lead letters by hand is slow and painful. Luckily for you, you travelled back in time with a laptop and a C compiler.



```
Fun fact: the words "uppercase" and "lowercase" come from typesetting.
Capital letters were stored in the upper case (a wooden drawer), small
letters in the lower one.
```

Chapter II

Common Instructions

- This exercise must be written in C.
- Your code must be written in accordance with the Norm. It will be checked with `norminette`, and you will receive a 0 if there is a Norm error.
- Your program must compile with `cc` and the flags `-Wall`, `-Wextra` and `-Werror`. If it does not compile, you will get 0.
- Your program must not quit unexpectedly (segmentation fault, bus error, double free, etc.). If this happens, your exercise will be considered non-functional and will receive a 0.
- Declaring global variables is strictly forbidden.
- You may only use the external functions listed in this subject. Using a forbidden function is considered cheating. Cheaters get -42, and this grade is non-negotiable.
- All your output must be done with `write`: `printf`, `fprintf`, `putchar` and the like are forbidden in every exercise.
- No Makefile is required: your program will be compiled with the command given in Chapter IV.
- We encourage you to test your program with many different inputs, including edge cases (empty string, very long text, spaces only...). Your evaluators will do it, for sure.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded.

Chapter III

AI Instructions

Context

This exercise is designed to help you consolidate the fundamental building blocks of your 42 training. To properly anchor these skills, you need to struggle with them yourself: true learning requires genuine intellectual effort, through challenge, repetition and peer-learning exchanges.

Main message

- ☛ Build strong foundations without shortcuts.
- ☛ The learning journey is more important than the result.
- ☛ An exercise solved by an AI is an exercise you did not solve.

Learner rules

- You should apply reasoning to your exercise, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You must be able to explain every line you submit.

✓ Good practice:

> I don't know how to detect the end of a paragraph. I take a sheet of paper, write the text five characters per line, and count. Then I compare my method with a peer's. It clicks.

✗ Bad practice:

> I paste the subject into an AI and copy its answer. During the evaluation, I can't explain why my program displays an empty line here. I fail. During the exam (no AI) I'm stuck again. I fail.

Chapter IV

Mandatory part

Turn-in directory	`ex00/`
Files to Submit	`ft_paragraph.c, ft_paragraph.h`
Compilation	`cc -Wall -Wextra -Werror ft_paragraph.c`
Arguments	`The text to lay out`
External Function	`write`
Description	`Lays out a text in lines and paragraphs, following the settings stored in a structure.`

IV.1 Technical considerations

✓ **Allowed:**

- The `write` function.
- Helper functions, within the limits of the Norm.
- The `t_config` structure given below, which you **must** use.

✗ **Forbidden:**

- Any other external function (`printf`, `puts`, `strlen`...).
- Global variables.
- Writing the layout settings anywhere else than in the initialization of the structure.

IV.2 The structure

Your master's three settings must live together, in the following structure. Copy it, as is, into a header file named `ft_paragraph.h`, protected against double inclusion, and include this header in your `ft_paragraph.c`:

```
typedef struct s_config
{
    int p_nbr;
    int p_len;
    int l_len;
}   t_config;
```



The Norm forbids typedef declarations in .c files: that is why the structure lives in a header.

Each field has a precise meaning:

- `p_nbr`: the number of paragraphs to display.
- `p_len`: the number of lines in each paragraph.
- `l_len`: the number of characters in each line.

Your `main` function must declare a variable of type `t_config` and fill it with these values:

```
config.l_len = 5;
config.p_len = 2;
config.p_nbr = 3;
```

Then, the whole layout must be done by the following function, which receives the text and the **address** of the structure:

```
void    ft_make_paragraph(char *input, t_config *config);
```



The values 5, 2 and 3 must appear only once in your code: when you fill the structure. Everywhere else, use the fields of the structure.

During the evaluation, these values will be changed. Your program must still work with any strictly positive values.

IV.3 Expected output

Your program takes exactly one argument: the text to lay out. It displays it on the standard output, following these rules:

- The characters of the text are displayed in their original order, `l_len` characters per line. Each line ends with a newline.
- Every character counts, spaces and punctuation included. Words can be cut anywhere: your master does not care about words, only about characters.
- Every time a paragraph is complete (`p_len` lines), an empty line is displayed.
- Once `p_nbr` paragraphs have been displayed, the layout is over.

If the number of arguments is not 1, your program displays the following message on the **error output**, followed by a newline, and returns 1:

Situation	**Message**
wrong number of arguments	`Error: usage ./a.out <text>.`

IV.4 Too much, or not enough

Too much text. If some characters are left once the last paragraph is complete, display `Rest: ` (with a space after the colon), followed by all the remaining characters and a newline.

Not enough text. If the text ends before the last paragraph is complete, first end the last line with a newline if it is incomplete. Then display the following messages, in this order, each followed by a newline:

Message	**Displayed...**
`Not enough paragraphs.`	always
`Missing line in paragraph.`	only if the text ends in the middle of a paragraph
`Missing char in line.`	only if the text ends in the middle of a line



These messages are not errors: they are part of the normal output of your program. They are displayed on the standard output, and your program returns 0.

IV.5 Examples

With the values given above (`l_len = 5`, `p_len = 2`, `p_nbr = 3`), the text fits exactly when it is 30 characters long:

```
$> ./a.out "abcdefghijklmnopqrstuvwxyz0123" | cat -e
abcde$
fghij$
$
klmno$
pqrst$
$
uvwxy$
z0123$
$
$> ./a.out "Lorem ipsum dolor sit amet, consectetur" | cat -e
Lorem$
 ipsu$
$
m dol$
or si$
$
t ame$
t, co$
$
Rest: nsectetur$
$>
```

When the text is too short:

```
$> ./a.out "Mainz 1450" | cat -e
Mainz$
 1450$
$
Not enough paragraphs.$
$> ./a.out "Printing press!" | cat -e
Print$
ing p$
$
ress!$
Not enough paragraphs.$
Missing line in paragraph.$
$> ./a.out "Gutenberg" | cat -e
Guten$
berg$
Not enough paragraphs.$
Missing line in paragraph.$
Missing char in line.$
$> ./a.out "" | cat -e
Not enough paragraphs.$
$> ./a.out
Error: usage ./a.out <text>.
$>
```

Chapter V

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer evaluation. Make sure to double-check the names of your files and folders to ensure they are correct.



`Your files must be placed in a folder named ex00 at the root of your repository: ex00/ft_paragraph.c and ex00/ft_paragraph.h`

During the evaluation, a brief **modification of the exercise** may be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

This step is meant to verify your actual understanding of the exercise. It should be feasible within a few minutes, in any development environment you choose.

For this exercise, your evaluator can for example change the values stored in the structure, or add a fourth setting to it (such as a character to display at the start of each line). If your code really relies on the structure, it will only take you a few seconds.